

1 Process Configuration

Processes by means of which one or more similar service starts may be performed can be configured in the MOC.

Please note: At present, such processes are exclusively used in the course of maintenance applications. The application generator creates standard processes for the activation of maintenance applications.

For this purpose, the processes support the following steps

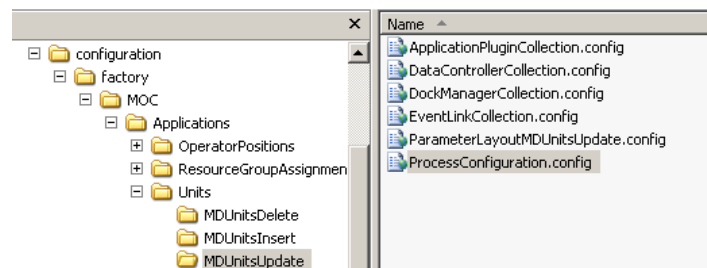
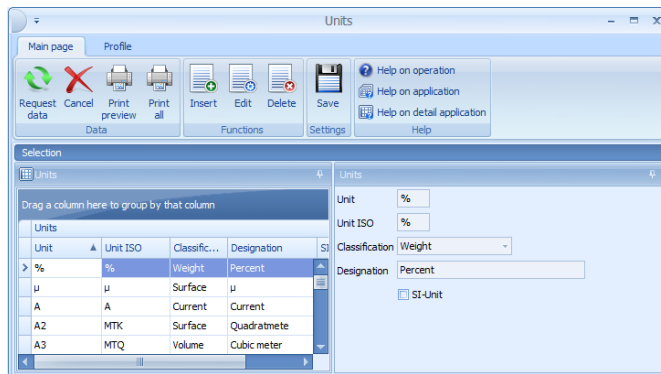
- Transfer of one or more parameter records from a data source. A parameter record describes the parameters to be transferred to the service. If more than one parameter record is transferred, the service is performed several times.
- Display of a dialog requesting the user to confirm the service activation (e.g. confirmation before activating DELETE services).
- Modification of a parameter record by a script or collection of a parameter record by a service activation.
- Activation of preparatory services (e.g. LOCK before UPDATE or NEW before INSERT).
- Processing or entry of parameter records in a dialog by the user (e.g. entry of values for INSERT services).
- Activation of a service with the parameter record.
- Activation of a script if the service returns an error (e.g. if a key value is missing) or general activation of a downstream service (e.g. UNLOCK for UPDATE services).

1.1 Processes

Each process requires one or more data sources provided with parameters from different parameter sources as a minimum requirement. For this purpose, an `ApplicationContainerForEdit`, via which the data sources and the relevant parameter mappings can be configured, is used as a basic component for each process.

This also applies to processes for services not requiring any explicit input dialog, e.g. the activation of DELETE services. As regards service activations with an input dialog (e.g. INSERT), this dialog is implemented as the plugin of an `ApplicationContainerForEdit` (usually a `LayoutApplication` plugin).

The process is configured by means of the `ProcessConfiguration` containing a specified number of `ProcessConfigurationItems`, which are each evaluated in relation to specified points. The `ProcessConfiguration` is a characteristic of the `ApplicationContainerForEdit` and is saved in a separate configuration file (`ProcessConfiguration.config`).



The figure shows the distribution of the different components of an application to the configuration files. In the example, the three functions "Insert", "Process" and "Delete" were additionally configured for the application "Units". The application is saved in the folder ".\MOC\Applications\Units". Each of the functions has its own `ApplicationContainerForEdit`, saved in a separate subfolder. In the figure, the subfolder "MDUnitsUpdate" has been selected, so that its configuration files are visible. The file "ProcessConfiguration.config" defines the process by means of which an update can be executed.

When the application generator is used, standard processes for Insert, Update, Delete and Copy are automatically predefined if necessary. At present, the configuration of other functions and/or the adaptation of the generated processes is implemented by editing the file `ProcessConfiguration.config` (there may be an editor for this in the future).

1.2 Structure of a process

At present, all processes have the same structure, i.e. they work according to the same pattern. A `ProcessConfiguration` consists of a list of `ProcessConfigurationItems`, run in a fixed sequence.

The process of an `ApplicationContainerForEdit` is started by the command `<container>.ProcessExecute`. Subsequently, the following steps are run through.

- Identification of the selected data in the `ParentController`, i.e. the "relevant" `DataController` of the activating `ApplicationContainer`. In general, the user will have highlighted one or more lines in a grid; these will be identified by this.

- Activation of `ProcessConfiguration.Prerequisite`, e.g. used to check whether the process is to be implemented.
- Identification of the number of steps - if more than one data record is selected, the process is run through the corresponding number of times.
- In each step
 - Activation of `ProcessConfiguration.StepParameterModification`, to check, modify or initially identify parameters (e.g. NEW Service).
 - Activation of `ProcessConfiguration.StepPrerequisite`, to check whether a process may be implemented or whether other preconditions (e.g. LOCK Service) are executed.
 - Depending on the type of process, either the dialog is opened (`ProcessTypes.Dialog`), or the process is implemented directly (`ProcessTypes.Direct`). In both cases, `ProcessConfiguration.StepExecute` is implemented; this is where the "main service" is activated.
 - Activation of `ProcessConfiguration.StepFinish`, in order to perform a follow-up treatment (e.g. UNLOCK Service).
- After all steps are completed, activation of `ProcessConfiguration.Finish`.

If exceptions are generated in the process, they are either intercepted at the end of the step loop, so that the user can decide whether to continue with the next steps, or the entire process is canceled.

1.3 Structure of `ProcessConfigurationItems`

The steps described above are each configured by `ProcessConfigurationItems`. The following characteristics can be defined for each item.

- **Type**: a value of `ProcessItemTypes`, e.g. `Prerequisite`, `StepExecute`, `StepFinish`. This value is automatically set to a `ProcessConfiguration` when an item is assigned and is primarily used for information and debugging purposes.
- **PreScriptName**: Name of a script to be executed before the execution of a service. If no name is indicated, the script is searched on the basis of the default name.
- **SingleItemText** and **MultiItemText**: Text (and/or language key) displayed in a Yes/No dialog before the service is activated. If the user clicks No, the entire process step is interrupted. If 0 or 1 parameter was transferred, the `SingleItemText` is displayed; if several parameters were transferred, the `MultiItemText` is displayed.
- **ServiceName**: Name of the service to be activated in this process step.
- **PostScriptName**: (Optional) name of a script provided by MPDV, to be executed before the execution of a service.

When a `ProcessConfigurationItem` is "executed", the above-stated characteristics are run through in the sequence presented, if they have been defined.

1.4 Special case: Activation of a process from another application

Some processes are needed within different applications, e.g. 'Set operation status' is to be activated from the order overview AND the operation maintenance. In this case, you do not want to define this process repeatedly and then have to update it at different locations in the case of modifications.

For this reason, it is also possible to activate a process already defined for an existing application from another application. For this purpose, the configuration has to be adapted as follows:

- Add a new button to the toolbar, by means of which the process can be activated.
- Function: `openApplicationContainerForEdit`
- Parameter: string with the following format: `id=<Value> parentSetting=<Value> callerController=<Value>`
 - o `id`=Name of the process to be executed
 - o `parentSetting`=application to which the process to be activated is associated
 - o `callerController`=main controller of the application from where this process is to be activated

Example: The process `operationsetStatus` of the application `OrderOverview` is to be activated from the screen `EditOperations`. In this case, the correct string for the parameter entry of the toolbar button would be:

`Id=operationsetStatus parentSetting=OrderOverview callerController=BOOperationList`

Important: It does not, of course, makes sense to use this randomly, but only if the data of the activating application indeed corresponds to that of the maintenance dialog.